Hydroelectric Power Plant Simulation Using OpenGL

An Interactive 3D Journey from the Reservoir to the Power Grid

Submitted by

Md. Tahmid Islam
Roll: 2107107

Submitted to

Md Tajmilur Rahman, Lecturer
Md. Mubtashim Abrar Nihal, Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

11 October, 2026

Abstract

This report describes *HYDRO*, a real-time 3D simulation of a hydroelectric power plant written in C++ and OpenGL 3.3. The project began as a proposal for a small "smart flood sluice gate". On review, the course teacher asked for the idea to be grown into a complete hydroelectric power plant, so the gate became one part of a much larger system. The final program follows a drop of water, and later the energy it carries, through eleven connected scenes: the reservoir, the trash rack, the intake gate, the penstock, the main inlet valve, a Francis turbine, the shaft, the generator, the step-up transformer, and finally the switchyard and power line that lead to the grid. At every step the viewer takes part: opening the gates, starting the turbine, exciting the generator and closing the breaker.

Every object is generated in code and every texture is calculated in a shader; no model files, image files or maths libraries are used. Apart from GLFW (window and input) and GLAD (loading OpenGL functions), all code, including the matrix library, the font and the screenshot writer, was written for this project. The program demonstrates every topic required by the course: Blinn–Phong lighting with directional and point lights, translation, rotation, scaling, composite and hierarchical transformations, switchable Gouraud and Phong shading, continuous physics-based animation, and mouse, keyboard and on-screen interaction. A Whitted-style ray tracer running on the GPU renders a glass penstock with exact ray–surface intersections, shadow rays, reflection, refraction and volume ray marching. Shadow mapping, procedural textures, water rendering, particle systems and a bloom post-process were added to give the scenes a cinematic look. The plant itself is driven by one consistent data sheet (53 m head, 180 m^3/s, 86 MW, 50 Hz), and a cause-and-effect model makes sure that nothing on screen happens before its cause. The finished program runs at 34–60 frames per second at 1600×900 on a GTX 1650 laptop GPU.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background . . . 1
1.2 From a Sluice Gate to a Power Plant . . . 1
1.3 Objectives . . . 2
1.4 Tools and Constraints . . . 2

2 Proposed and Implemented Features **3**
2.1 Course Requirements . . . 3
2.2 Proposed Objects . . . 3
2.3 Proposed Features . . . 4
2.4 New Features Added for the Power Plant . . . 4

3 System Design **6**
3.1 Architecture . . . 6
3.2 The Story: Eleven Connected Scenes . . . 6
3.3 The Frame Pipeline . . . 7
3.4 The Plant Model . . . 8

4 Graphics Theory and Implementation **10**
4.1 Geometry and Procedural Modelling . . . 10
4.2 Transformations . . . 10
4.3 Camera System . . . 12
4.4 Lighting . . . 13
4.5 Gouraud and Phong Shading . . . 14
4.6 Shadow Mapping . . . 15
4.7 Procedural Textures . . . 15
4.8 Water Rendering . . . 16
4.9 Ray Tracing . . . 17
4.10 Animation and Physics . . . 19
4.11 Post-Processing and Transitions . . . 20
4.12 User Interface and Text . . . 21
4.13 Interaction . . . 21

5 Objects of the Scene **22**
5.1 Terrain, Reservoir and River . . . 23
5.2 Dam and Spillway . . . 23
5.3 Intake Structure, Trash Rack and Debris . . . 23
5.4 Intake Gate and Penstock . . . 23
5.5 Powerhouse . . . 23
5.6 Main Inlet Valve . . . 23
5.7 Francis Turbine . . . 23
5.8 Generator . . . 24
5.9 Bus Ducts and Step-up Transformer . . . 24
5.10 Switchyard and Power Line . . . 24
5.11 Summary of Moving Objects . . . 24

6 Scene-by-Scene Walkthrough **25**
6.1 Scene 1: Preview of the Finished Plant . . . 25
6.2 Scenes 2 and 3: Reservoir, Trash Rack and Intake Gate . . . 25
6.3 Scenes 4 and 5: Penstock and Ray-Traced Pipe . . . 26
6.4 Scenes 6 and 7: Main Inlet Valve and Francis Turbine . . . 26
6.5 Scenes 8 and 9: Shaft and Generator . . . 27
6.6 Scenes 10 and 11: Transformer and Grid . . . 27

7 Results, Testing and Discussion **29**
7.1 Main Results . . . 29
7.2 Testing Method . . . 29
7.3 Performance . . . 29
7.4 Limitations . . . 29

8 Conclusion and Future Work **31**
8.1 Conclusion . . . 31
8.2 Future Work . . . 31

References **32**

A The Approved Proposal **33**

B Building and Running the Program **34**

Chapter 1

Introduction

1.1 Background

Bangladesh depends on rivers, and most of us have seen sluice gates and embankments holding back water during the monsoon. A hydroelectric power plant takes the same idea much further: instead of only holding water back, it lets the water fall through a turbine in a controlled way and turns the energy of that fall into electricity. The path from the lake behind a dam to the socket in a home is long, and most of it is hidden inside concrete and steel. That makes it a good subject for computer graphics, where we can cut objects open, slow time down and show invisible things such as magnetic fields and electric current.

The project tells this story as a live OpenGL program that walks through the plant in the same order as the water. Instead of only watching, the viewer opens the gates, starts the turbine, excites the generator and closes the breaker. Figure 1.1 shows the plant as it appears when the program starts.

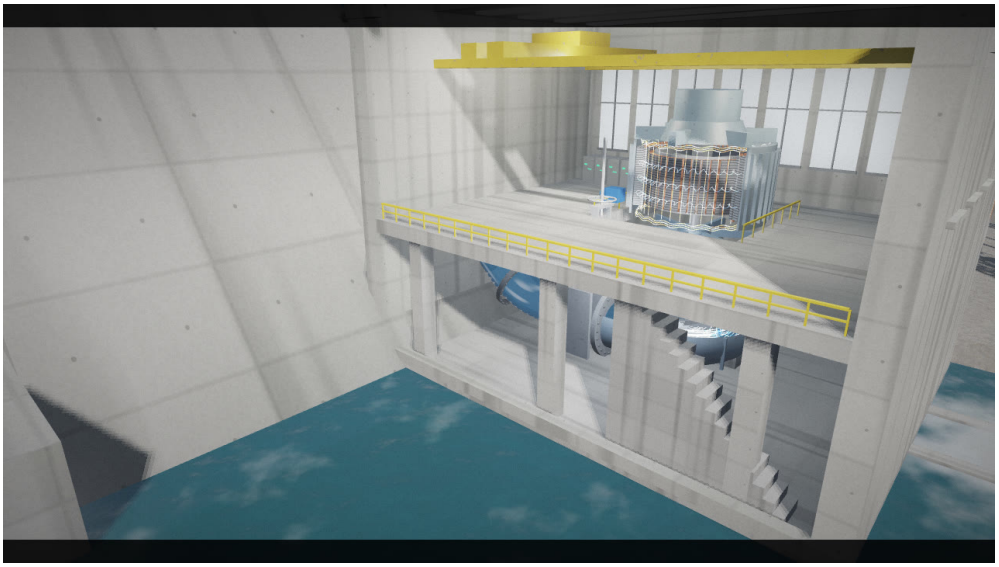


Figure 1.1: The running plant: spillway on the left, cut-away powerhouse with the generator floor and the turbine pit below it.

1.2 From a Sluice Gate to a Power Plant

The original proposal (Appendix A) was titled *Smart Flood Sluice Gate*. It described a canal, a movable metal gate, a rotating control wheel, a changing water level, a floating buoy and warning lights, rendered with Phong lighting. When the proposal was reviewed, the teacher wrote *"Hydroelectric Power Plant"* on it and asked for the project to become a hydroelectric plant simulation.

The two ideas fit together well, because a hydroelectric plant is full of gates, wheels, shafts and moving water. Every object and feature of the sluice-gate proposal therefore has a natural place in the power plant: the sluice gate becomes the intake gate and the spillway gates, the control wheel becomes the hand-wheel of the main inlet valve, the connecting shaft becomes the turbine–generator shaft, and the buoy becomes the floating debris stopped by the trash rack. Throughout this report the *hydroelectric power plant* is treated as the proposed project, and Chapter 2 compares it item by item with what was built.

1.3 Objectives

The main objectives of the project were:

1. Build an interactive 3D model of a complete hydroelectric power plant using only C++, OpenGL and GLSL, without model files or maths libraries.
2. Demonstrate the course topics clearly: lighting, transformations, Gouraud and Phong shading, motion and animation, user interaction, and ray tracing.
3. Follow the real order of energy conversion, from potential energy in the reservoir to electrical energy in the grid, and keep cause and effect correct.
4. Give the program a clear, cinematic look: smooth camera moves, cut-away views, glowing flow lines and labelled parts.
5. Keep the code readable, with one file for each object and each scene, so that the project is easy to explain and extend.

1.4 Tools and Constraints

The course asked for plain OpenGL in C or C++, without Python and without third-party libraries. Table 1.1 lists everything the project uses. GLFW and GLAD are the only external code: GLFW opens the window and reads the keyboard and mouse, and GLAD looks up the OpenGL functions at run time, which Windows requires for anything newer than OpenGL 1.1. Neither of them draws anything. Vectors, matrices, geometry, textures, the font and the image writer are all part of the project.

Table 1.1: Tools used in the project.

Tool	Purpose
C++17	Main language (about 8 000 lines in 85 source files).
OpenGL 3.3 Core	All rendering [17].
GLSL 3.30	21 shader files [18] (about 1 300 lines): lighting, water, ray tracer, post-processing, user interface.
GLFW 3	Creates the window and the OpenGL context, reports keyboard, mouse and time.
GLAD	Generated loader for OpenGL function pointers.
MinGW-w64 g++ / CMake	Compiling on Windows (`build.bat` or CMake).
Not used: GLM, image loaders, model loaders, physics engines, game engines, Python.	

Chapter 2

Proposed and Implemented Features

This chapter answers a simple question: *was everything that was promised actually built?* The comparison uses two labels. **Completed** means the proposed item is present and works as described. **Extended** means the item is present and goes clearly beyond the original description, or that it is a new feature added for the hydroelectric plant.

2.1 Course Requirements

The course set six compulsory topics. Table 2.1 shows where each one is implemented and how it can be shown during a demonstration.

Table 2.1: Compulsory course topics and where they are implemented.

Topic	Status	Implementation	How to show it
Lighting	**Extended**	Blinn–Phong with hemisphere ambient, a directional sun, up to four point lights, Fresnel reflections, shadow mapping and under-water light (`shaders/common.glsl`).	Any scene; `L` turns shadows on and off.
Transformation	**Completed**	Own matrix library (`MathLib.h`): translation, rotation, scaling, composite and hierarchical transforms on gates, valve, 24 guide vanes, runner, rotor, fans and debris.	Scenes 3, 6 and 7.
Gouraud & Phong shading	**Completed**	Two separate shader pairs: `gouraud.vert/.frag` (per vertex) and `lit.vert/.frag` (per pixel).	`G` cycles Mixed, Phong and Gouraud.
Motion / animation	**Extended**	Physics-based water fronts, turbine inertia, buoyant debris, particles, glowing flow lines and spline camera paths, all driven by frame time.	Every scene; `[` `]` change the speed.
Interaction	**Extended**	On-screen buttons with keyboard shortcuts, mouse orbit, pan and zoom, speed control, and buttons that stay locked until their cause has happened.	Every scene.
Ray tracing	**Extended**	Whitted-style GPU ray tracer with exact intersections, shadow rays, reflection, refraction and volume ray marching (`shaders/raytrace.frag`).	Scene 5; `UP`/`DOWN` change the head.

2.2 Proposed Objects

Table 2.2 maps each object in the proposal to its form in the power plant.

Table 2.2: Objects of the proposal and their implementation in the power plant.

Proposed object	Status	Implemented as
Canal base and side walls	**Extended**	The water channels of the plant: the reservoir basin, the intake channel, an 11 m wide conduit through the dam, the penstock and the river bed. The valley is a 260×260 height-field terrain.
Movable metal sluice gate	**Extended**	A vertical-lift intake gate with hydraulic cylinders, three spillway gates in the dam crest, and the gate of the main inlet valve.
Gate frame and vertical support pillars	**Extended**	The intake structure with side piers and a top deck, the gate shaft, the spillway piers, and the steel frame of the trash rack.

Proposed object	Status	Implemented as
Rotating control wheel and connecting shaft	**Extended**	The hand-wheel of the main inlet valve on its stem, and the main vertical shaft that joins the turbine runner to the generator rotor, with a guide bearing and a bolted coupling flange.
Water surface and water-level indicator	**Extended**	Animated water in the reservoir, river, conduit, penstock and spiral casing, plus live read-outs of gate opening, conduit level and casing filling.
Floating buoy for showing water movement	**Completed**	Floating debris (a branch, logs, crumpled plastic, straw and leaves) that bobs on the water and drifts with the current until the trash rack stops it.
Warning beacon and light fixture	**Completed**	Lamps in the powerhouse (point lights) and red and green status lamps on the circuit breakers.
Ground area and simple control platform	**Extended**	The valley terrain with boulders, the powerhouse floors with yellow railings, the switchyard platform, and the on-screen control panel.
Directional and point light sources	**Completed**	The sun as a directional light and the powerhouse lamps as point lights with distance fall-off.

2.3 Proposed Features

Table 2.3 maps each feature in the proposal to its implementation.

Table 2.3: Features of the proposal and their implementation.

Proposed feature	Status	Implemented as
The gate moves vertically to control water flow	**Extended**	The intake gate rises with its piston rods; water jets under it with Torricelli speed $\sqrt{2gh}$ and fills the conduit only while it is open. The spillway gates and the inlet valve work the same way.
The control wheel rotates with its connected shaft	**Extended**	The valve hand-wheel turns only while the valve gate is moving. The runner, shaft and generator rotor share one rotation angle, so they turn together.
The water level rises and falls dynamically	**Extended**	The conduit level, the penstock water front and the spiral casing filling are simulated every frame; closing a gate drains the system again.
Floating objects move up and down with the water	**Completed**	Each piece of debris is held at its floating depth by a spring–damper buoyancy model and bobs with a sine wave.
Translation, rotation, scaling, composite and hierarchical transformations	**Completed**	All five are used and explained in Section 4.2.
Phong shading with ambient, diffuse and specular reflection	**Extended**	Per-pixel Blinn–Phong, a switchable Gouraud mode, soft shadows, Fresnel reflection and tone mapping.
User control of the gate, animation speed, lighting and camera	**Extended**	Scene buttons, speed 0.25× to 2×, shadow, shading and bloom toggles, and an orbit, pan and zoom camera on top of the cinematic camera.
Reusable primitive meshes keep the scene optimised	**Extended**	A `MeshBuilder` creates every shape from a few primitives; one mesh is drawn many times with different matrices (24 guide vanes, three spillway gates), and hidden parts are culled.

2.4 New Features Added for the Power Plant

The features in Table 2.4 were not in the sluice-gate proposal. They were added to turn it into the hydroelectric plant the teacher asked for, and they are all marked **Extended**.

Table 2.4: Features added beyond the proposal.

New feature	Status	Summary
Complete plant in 11 scenes	**Extended**	Reservoir to grid in one continuous world with flying camera transitions (Section 3.2).
GPU ray tracing	**Extended**	Whitted ray tracer for the glass penstock (Section 4.9).
Shadow mapping	**Extended**	2048×2048 depth map with 3×3 PCF soft edges.
Gouraud shading mode	**Extended**	A second lighting path evaluated per vertex, switchable at run time.
Procedural textures	**Extended**	15 material patterns from noise: concrete, steel, copper, rock, bark and more.
Water rendering	**Extended**	Sine-sum waves, Fresnel reflection, Snell's window, caustics and under-water fog.
Particle systems	**Extended**	Six systems for spray, mist, foam, splashes, gate spray and under-water dust.
Flow-line visualisation	**Extended**	Glowing pulses for water streamlines, magnetic field, current and the power line.
Turbine, generator, transformer, switchyard	**Extended**	Detailed machines with working cause and effect.
Physically based plant data	**Extended**	$P = \rho g Q H \eta$, $f = pn/120$ and other formulas drive every number on screen.
Cinematic cameras	**Extended**	Catmull–Rom camera paths with continuity between scenes.
Post-processing	**Extended**	Bloom, vignette and film grain; cross-dissolves between render styles.
Information display	**Extended**	Telemetry panel, 3D pointer labels, an oscilloscope and an energy summary, drawn with a hand-made pixel font.
Cut-away and X-ray views	**Extended**	Parts of the dam, intake, pipe, valve, casing and generator open up for viewing.
Testing tools	**Extended**	Scripted screenshots, an automatic play-through test and a benchmark mode.

In short

All six compulsory topics are present, every object and feature of the proposal has been implemented in the hydroelectric plant, and fifteen new features were added on top.

Chapter 3

System Design

3.1 Architecture

The program is organised in layers (Figure 3.1). `main.cpp` reads the command line and starts the `App`. The `App` owns the window, the input, the scene list, the transitions and the user interface. Each of the eleven *scenes* is a small class that only decides what the camera does, which buttons are offered and which labels are shown. All the actual content lives in a single `World` object, which builds the whole plant once and runs the simulation. The world is made of *objects*, one file per object, and the objects draw themselves through the `Renderer`, which talks to the GLSL shaders.

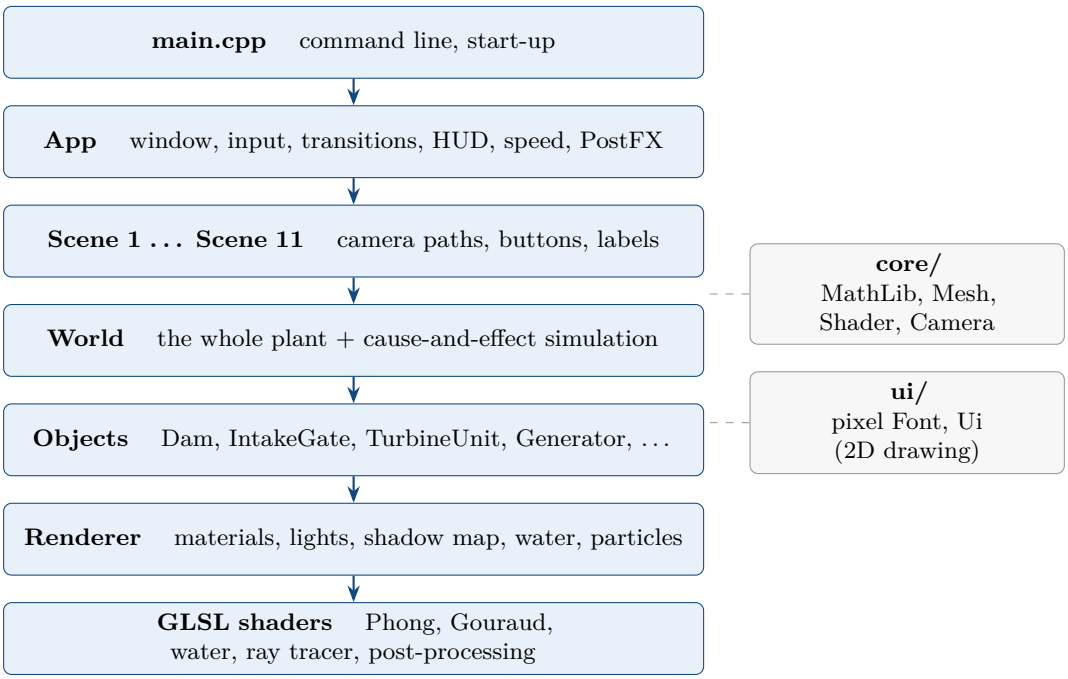


Figure 3.1: Layered architecture of the program.

Because every scene looks at the same `World`, nothing is duplicated. The water the viewer releases at the intake gate is the same water that later fills the spiral casing, and the turbine that spins in the preview is the same turbine that is examined in scene 7. Table 3.1 lists the main modules.

3.2 The Story: Eleven Connected Scenes

The program follows the real path of energy through the plant. Water carries potential energy from the reservoir, turns it into kinetic energy in the penstock and mechanical energy in the turbine, and the generator turns that into electrical energy, which the transformer and the switchyard send to the grid (Figure 3.2).

The scenes are not separate levels. When the viewer presses a *follow* button, the current camera position is handed to the next scene (`App::continueTo`), and the next camera path starts exactly there, so the camera keeps flying. Only where the rendering style changes completely (from under water to a cut-away diagram, or from rasterisation to ray tracing) does the program cross-dissolve instead. Table 3.2 lists the scenes and their controls.

Table 3.1: Main modules and their responsibilities.

Module	Responsibility
`core/MathLib.h`	Vectors, 4×4 matrices, all transformation and camera matrices.
`core/Mesh.cpp`	GPU buffers (VAO, VBO, EBO) and the `MeshBuilder` that generates shapes.
`core/Shader.cpp`	Compiles shaders and expands a simple `#include` so shaders can share code.
`core/Camera.cpp`	Look-at camera, user orbit and Catmull–Rom camera paths.
`render/Renderer.cpp`	Lights, materials, shadow pass, choice of Phong or Gouraud shader.
`render/ParticleSystem.cpp`	Particle update and camera-facing billboards.
`render/PostFX.cpp`	Bloom, vignette and film grain.
`render/FrameCapture.cpp`	Keeps the last frame for cross-dissolves.
`objects/World.cpp`	Builds the plant, runs the simulation, draws in the right order, culls.
`objects/PlantData.h`	The plant's data sheet and formulas.
`objects/*.cpp` (18 files)	One file per object (Chapter 5).
`scenes/*.cpp` (11 files)	One file per scene (Chapter 6).
`ui/Font.cpp`, `ui/Ui.cpp`	Hand-made 5×7 pixel font and 2D drawing of the HUD.

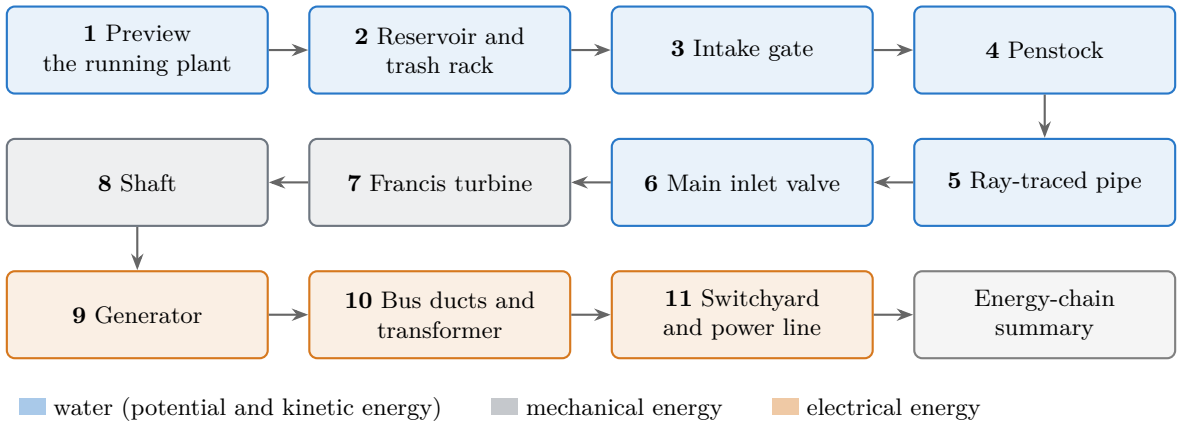


Figure 3.2: The eleven scenes follow the energy through the plant.

3.3 The Frame Pipeline

Every frame goes through the same steps (Figure 3.3). The scene first advances its story by $\Delta t \times$ speed, where Δt is the measured frame time, limited to 0.05 s so that a paused window cannot produce one huge physics step. The world is then drawn twice: once from the sun into the shadow map, and once from the camera into an off-screen colour texture. That texture goes through the post-processing passes, is copied for a possible dissolve, and finally the HUD is drawn on top.

Inside the main pass the order matters. The sky is drawn first, then all opaque objects with depth writing on, then transparent objects (water, glass, glowing lines, particles) with blending and depth writing off, so that they blend over what is already behind them.

Table 3.2: The eleven scenes and their controls.

#	Scene	What happens	Controls
1	Preview	The finished plant at work: spillway, cut-away powerhouse, spinning machines.	Stop/start water `SPACE`, show water movement `ENTER`
2	Reservoir and trash rack	Camera flies over the dam and dives to the intake; trash piles up at the rack.	Add more trash `T`, follow
3	Intake gate	The dam block lifts away; the gate opens and the conduit fills.	Open gate `O`; follow unlocks when fully open
4	Penstock	Water rushes down the pipe and stops at the closed valve.	X-ray `X`, replay flow, follow
5	Ray-traced pipe	Ray-traced glass penstock with glowing water.	Head `UP` `DOWN`, follow
6	Main inlet valve	The valve opens and the spiral casing fills.	Open valve `O`; follow unlocks when full
7	Francis turbine	Guide vanes open and the runner speeds up to 150 rpm.	Guide vanes `O`, section `X`; follow unlocks at 140 rpm
8	Shaft	Hologram-style rise along the turning shaft.	Automatic
9	Generator	Exciting the rotor creates a field and three-phase current.	Excite rotor `O`; follow unlocks at 10.5 kV
10	Transformer	Bus ducts carry 11 kV; the transformer raises it to 220 kV.	X-ray `X`, follow
11	Switchyard and grid	Closing the breaker sends power along the line; summary appears.	Breaker `O`, replay journey

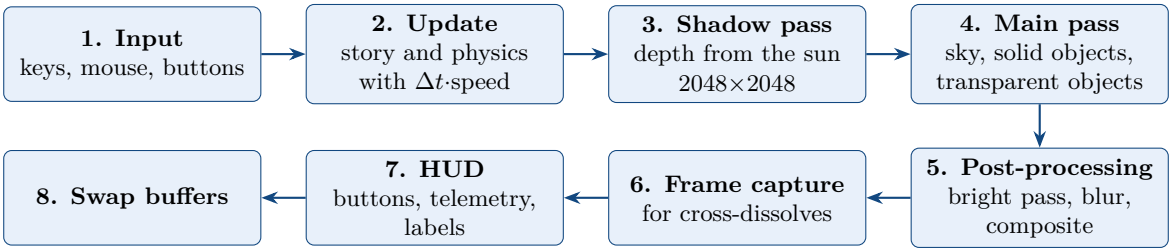


Figure 3.3: Steps of one frame.

3.4 The Plant Model

3.4.1 One data sheet

Early versions showed numbers that did not agree with each other: a flow that did not match the power, or an rpm that did not match the frequency. The final program takes every number from one data sheet, `objects/PlantData.h` (Table 3.3), so all read-outs are consistent.

3.4.2 Plant equations

The hydraulic power available from water falling through a head H is

$$P = \rho\, g\, Q\, H\, \eta = 1000 \times 9.81 \times 180 \times 53 \times 0.92 \approx 86.1 \text{ MW}, \tag{3.1}$$

where ρ is the density of water and g the gravitational acceleration. The torque on the shaft follows from $P = T\omega$:

$$T = \frac{P}{\omega} = \frac{P}{2\pi n/60} = \frac{86 \times 10^6}{15.71} \approx 5.5 \text{ MN m}. \tag{3.2}$$

Table 3.3: The plant data sheet.

Quantity	Value	Comment
Reservoir level, tail-water level	56 m, 3 m	world coordinates
Gross head H	53 m	$56 - 3$
Flow through one unit Q	180 m^3/s	at full guide-vane opening
Overall efficiency η	0.92	
Rated power P	86 MW	$\rho g Q H \eta$
Turbine and generator speed n	150 rpm	
Generator poles p	40	
Frequency f	50 Hz	$p\,n/120$
Generator voltage, grid voltage	11 kV, 220 kV	transformer ratio 1 : 20

A synchronous generator with p poles turning at n rpm produces [20]

$$f = \frac{p\,n}{120} = \frac{40 \times 150}{120} = 50 \text{ Hz}. \tag{3.3}$$

For three-phase power the line current is $I = P/(\sqrt{3}\,V)$, which gives 4.5 kA at 11 kV and only 0.23 kA at 220 kV. This is why the transformer steps the voltage up: the same power travels with twenty times less current, and the resistive loss I^2R falls by a factor of four hundred. The ideal speed of water falling through the full head is $v = \sqrt{2gH} \approx 32$ m/s.

The 3D rotor turns more slowly than 150 rpm so that the eye can follow it. The program keeps a visual angular speed ω_v and converts it back for the read-outs with $n = 150\,\omega_v/\Omega_v$, where $\Omega_v = 2.4$ rad/s is the visual speed at full load.

3.4.3 Cause and effect

A rule set for the whole project was that *nothing happens before its cause*. `World::update` enforces the chain in Table 3.4. Because of it, the viewer cannot reach a state that would be physically wrong, such as voltage from a stopped rotor or power flowing through an open breaker.

Table 3.4: Cause-and-effect rules in the simulation.

Effect	Requires
Conduit fills	intake gate open
Penstock fills	conduit full
Spiral casing fills	main inlet valve more than 30 % open and penstock full
Water flows through the runner	casing full and valve open; the flow equals the guide-vane opening
Runner speeds up	water flowing; speed follows the flow with inertia
Voltage appears	$V = 11\,\mathrm{kV} \times (\text{field current}) \times \min(1, n/150)$
Power line becomes live	breaker closed and $V > 9$ kV
Power is delivered	breaker closed; $P = \rho g (Q\,q) H \eta$ with q the opening

When the viewer jumps straight to a later scene with the number keys, `World::completeUpTo` performs every earlier step instantly, so the scene still starts in a correct state.

Chapter 4

Graphics Theory and Implementation

This chapter explains each graphics feature in three parts: the idea, the equations behind it, and how the project implements it. General background comes from Foley et al. [1] and *Real-Time Rendering* [2].

4.1 Geometry and Procedural Modelling

No model files are loaded. Every object is built at start-up by a `MeshBuilder` (`core/Mesh.cpp`) that knows a small set of primitives:

- **quads and boxes** for walls, floors, beams and tanks;
- **cylinders and cones** with optional caps and partial angles (for cut-aways), plus a helper that stands a cylinder between two points;
- **spheres, tori, disks and annuli** for bearings, bolts, rings and flanges;
- **grids** for terrain and water surfaces;
- **sweeps**: a circle swept along a path with rounded corners, used for the penstock, the spiral casing, bus ducts and flow lines;
- **extruded outlines**: a 2D profile pushed along an axis, used for the cross-section of the gravity dam.

Each vertex stores a position, a normal and a texture coordinate (32 bytes). The builder holds a current transformation matrix $\mathbf{M}$: every shape added is multiplied by $\mathbf{M}$ and its normals by the normal matrix of $\mathbf{M}$. This means a complex part can be assembled by placing simple shapes, exactly as a modeller would. For example, the 72 stator slots of the generator are one slot shape added 72 times with $\mathbf{M} = \mathbf{T}(\text{centre})\,\mathbf{R}_y(2\pi k/72)$. Texture coordinates are measured in metres, so procedural textures have the same scale on every object.

The finished vertices are uploaded once into a vertex buffer and an index buffer. Meshes that change every frame (the conduit water surface) are created with `GL_DYNAMIC_DRAW` and updated with `glBufferSubData`.

4.2 Transformations

4.2.1 Homogeneous matrices

Points are written in homogeneous coordinates $\mathbf{p} = (x, y, z, 1)^T$ so that translation, rotation and scaling can all be expressed as 4×4 matrices and combined by multiplication [1]. The matrix library `core/MathLib.h` stores them in column-major order, which is the layout OpenGL expects. The basic matrices are

$$\mathbf{T}(t) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{S}(s) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{4.1}$$

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{4.2}$$

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{4.3}$$

Rotation about an arbitrary unit axis $\mathbf{a}$ uses Rodrigues' formula,

$$\mathbf{R}_{\mathbf{a}}(\theta) = \cos\theta\,\mathbf{I} + (1-\cos\theta)\,\mathbf{a}\mathbf{a}^T + \sin\theta\,[\mathbf{a}]_\times, \tag{4.4}$$

where $[\mathbf{a}]_\times$ is the cross-product matrix of $\mathbf{a}$. A helper `alignY` builds an orthonormal basis whose y axis points along a given direction; it is used to stand cylinders between two points, for example bolts, pipes and insulators.

4.2.2 Composite and hierarchical transformations

A model matrix is usually a product of several basic matrices. With column vectors the right-most matrix acts first, so

$$\mathbf{M} = \mathbf{T}\,\mathbf{R}\,\mathbf{S} \tag{4.5}$$

first scales the object, then rotates it, and finally moves it into place. In a *hierarchy*, a child part is defined relative to its parent, and its world matrix is the parent's matrix times its own local matrix: $\mathbf{M}_{\text{child}} = \mathbf{M}_{\text{parent}}\,\mathbf{M}_{\text{local}}$. Moving the parent automatically moves every child. Table 4.1 lists the transformations used in the plant.

Table 4.1: Transformations used on the moving parts.

Part	Matrix	Type
Spillway gates (3)	$\mathbf{T}(\text{bay}_i + h\,\hat{\mathbf{y}})$	one mesh, three translations
Dam cut-away block	$\mathbf{T}(0,\ 160\,e(\text{lift}),\ 0)$	translation with easing
Intake gate and rods	$\mathbf{M}_{\text{base}}\,\mathbf{T}(0, h, 0)$	hierarchical: rods follow the gate
Valve gate and stem	$\mathbf{T}(0,\ 4.4\,u,\ 0)$	translation
Valve hand-wheel	$\mathbf{T}(\text{top})\,\mathbf{R}_y(\phi)$	composite
Guide vanes (24)	$\mathbf{T}(\mathbf{p}_k)\,\mathbf{R}_y(-(\theta_k + 90^\circ + \beta))$	composite; each vane turns about its own pivot
Runner, shaft, rotor	$\mathbf{T}(\mathbf{c})\,\mathbf{R}_y(\alpha)$	composite; one shared angle
Transformer fans (4)	$\mathbf{T}(\mathbf{f}_i)\,\mathbf{R}_z(\phi)$	composite
Floating debris	$\mathbf{P}\,\mathbf{T}(\mathbf{x})\,\mathbf{R}_y\mathbf{R}_z\mathbf{R}_x\,\mathbf{S}(s)$	hierarchical, with scaling

The intake gate is the clearest example of a hierarchy. The gate leaf and the piston rods that lift it are drawn with the same matrix, so the rods slide into their fixed cylinders as the gate rises:

```
// IntakeGate::draw - the rods are attached to the leaf, so they use its matrix
Mat4 up = base * translate(Vec3(0, lift, 0));
r.drawMesh(leaf, up, paint);
r.drawMesh(rods, up, steel);        // rods move WITH the gate
r.drawMesh(cylinders, base, paint); // cylinders stay where they are
```

The debris uses a two-level hierarchy. Each piece is simulated in a local "flow frame" in which the current always runs along $+x$. The parent matrix $\mathbf{P}$ then places that frame at the trash rack in the

world, so the physics code stays simple.

The 24 guide vanes show a composite transformation clearly. All vanes share one mesh. Vane k is moved to its pivot $\mathbf{p}_k$ on a circle of radius 1.98 m and turned about the vertical axis by its position angle $\theta_k = 2\pi k/24$ plus the opening angle $\beta = 8^\circ + 38^\circ \times \text{opening}$. Changing one number, the opening, turns all 24 vanes together, like the operating ring of a real turbine.

4.2.3 Normal transformation

Normals cannot simply be multiplied by the model matrix when the matrix contains non-uniform scaling, because they would no longer be perpendicular to the surface. The correct normal matrix is the inverse transpose of the upper 3×3 part,

$$\mathbf{N} = \left(\mathbf{M}_{3\times 3}^{-1}\right)^{T}, \tag{4.6}$$

which `normalMatrix` computes with the adjugate formula.

4.2.4 Viewing and projection

A point goes from object space to clip space through

$$\mathbf{p}_{\text{clip}} = \mathbf{P}\,\mathbf{V}\,\mathbf{M}\,\mathbf{p}_{\text{local}}. \tag{4.7}$$

The view matrix $\mathbf{V}$ is built by `lookAt`. From the eye $\mathbf{e}$, the target $\mathbf{t}$ and the world up vector, it forms the forward vector $\mathbf{f} = \text{normalize}(\mathbf{t} - \mathbf{e})$, the side vector $\mathbf{s} = \text{normalize}(\mathbf{f} \times \mathbf{up})$ and the true up vector $\mathbf{u} = \mathbf{s} \times \mathbf{f}$, and uses them as the rows of a rotation followed by a translation by $-\mathbf{e}$. The perspective matrix, for a vertical field of view φ, aspect ratio a and near and far planes n and f, is

$$\mathbf{P} = \begin{bmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}, \qquad c = \frac{1}{\tan(\varphi/2)}. \tag{4.8}$$

The shadow pass uses an orthographic matrix instead, because sunlight is parallel.

4.3 Camera System

The camera has two layers. The *director's camera* is scripted by each scene as a list of key positions $(t_i, \mathbf{e}_i, \mathbf{t}_i)$, and the eye and target are interpolated with a Catmull–Rom spline [10]. For the segment between keys $\mathbf{P}_1$ and $\mathbf{P}_2$ with neighbours $\mathbf{P}_0$ and $\mathbf{P}_3$,

$$\mathbf{C}(u) = \tfrac{1}{2}\Big[2\mathbf{P}_1 + (\mathbf{P}_2 - \mathbf{P}_0)\,u + (2\mathbf{P}_0 - 5\mathbf{P}_1 + 4\mathbf{P}_2 - \mathbf{P}_3)\,u^2 + (-\mathbf{P}_0 + 3\mathbf{P}_1 - 3\mathbf{P}_2 + \mathbf{P}_3)\,u^3\Big], \quad u \in [0,1]. \tag{4.9}$$

The curve passes through every key and has a continuous tangent, so the camera glides through the keys like a real crane shot instead of stopping at each one. On the first and last segments u is replaced by $u' = \frac{1}{2}(3u^2 - 2u^3) + \frac{1}{2}u$, which eases the motion in and out.

The second layer is the *user orbit*. Dragging the mouse changes a yaw ψ and a pitch ϑ around the target, the wheel changes a zoom factor z, and the keyboard adds a pan vector. The final eye position is

$$\mathbf{e}' = \mathbf{t} + \mathbf{pan} + z\,d\,(\sin\psi\cos\vartheta,\ \sin\vartheta,\ \cos\psi\cos\vartheta), \tag{4.10}$$

with the pitch limited to $[-80^\circ, 85^\circ]$. Pressing C resets the orbit and returns to the director's camera.

4.4 Lighting

4.4.1 The Phong reflection model

The Phong model [3] splits the light reflected from a surface into three parts. *Ambient* light is a constant that stands for light bouncing around the scene. *Diffuse* light depends on how directly the surface faces the light, following Lambert's cosine law. *Specular* light is the shiny highlight that appears where the reflection of the light points towards the eye. With the unit normal $\mathbf{N}$, the direction to the light $\mathbf{L}$, the direction to the viewer $\mathbf{V}$ and the reflected light direction $\mathbf{R} = 2(\mathbf{N}\cdot\mathbf{L})\mathbf{N} - \mathbf{L}$,

$$I_a = k_a\, I_{\text{amb}}, \tag{4.11}$$

$$I_d = k_d\, I_L\, \max(\mathbf{N}\cdot\mathbf{L},\, 0), \tag{4.12}$$

$$I_s = k_s\, I_L\, \max(\mathbf{R}\cdot\mathbf{V},\, 0)^n, \tag{4.13}$$

$$I = I_a + \sum_{\text{lights}} \left(I_d + I_s\right), \tag{4.14}$$

where k_a, k_d, k_s are the material's ambient, diffuse and specular coefficients, I_L is the light's colour and n is the shininess: a larger n gives a smaller, sharper highlight.

4.4.2 Blinn–Phong as used in the project

The project uses Blinn's variant [4], which replaces $\mathbf{R}\cdot\mathbf{V}$ with $\mathbf{N}\cdot\mathbf{H}$, where $\mathbf{H}$ is the halfway vector between the light and the eye:

$$\mathbf{H} = \frac{\mathbf{L}+\mathbf{V}}{\|\mathbf{L}+\mathbf{V}\|}, \qquad I_s = k_s\,\frac{n+8}{32}\, I_L\, \max(\mathbf{N}\cdot\mathbf{H}, 0)^n. \tag{4.15}$$

The highlight looks more natural at grazing angles and is cheaper to compute. The factor $(n+8)/32$ is a normalisation, scaled so that $n = 24$ gives a factor of one: it keeps a sharp highlight bright and a broad one dim, so changing the shininess does not change the total amount of reflected light much. The complete lighting function, `shade()` in `shaders/common.glsl`, adds four more terms to Equation (4.14):

- **Hemisphere ambient.** Instead of a flat constant, the ambient colour blends from the ground colour to the sky colour according to which way the surface faces: $I_a = k_d\, I_{\text{amb}} \cdot \operatorname{mix}\!\left(C_{\text{ground}}, C_{\text{sky}}, \frac{N_y+1}{2}\right)$. Surfaces facing up look bluer and surfaces facing down look warmer.
- **Shadows.** The sun's diffuse and specular terms are multiplied by a shadow factor $S \in [0,1]$ from the shadow map (Section 4.6).
- **Point lights.** The powerhouse lamps fall off with distance d as $\text{att}(d) = 1/(1 + 0.06\, d^2)$.
- **Environment reflection.** Metal and wet surfaces reflect the sky. The strength follows Schlick's approximation of the Fresnel equations [9], $F = r + (1-r)(1-\mathbf{N}\cdot\mathbf{V})^5$, so reflections become stronger at grazing angles.

Together, the colour of a surface with albedo $\mathbf{c}$ is

$$\mathbf{C} = \mathbf{c}\, I_a + S\, I_{\text{sun}}[\mathbf{c}\, \max(\mathbf{N}\cdot\mathbf{L}, 0) + I_s] + \sum_j \text{att}(d_j)\, I_j[\mathbf{c}\, \max(\mathbf{N}\cdot\mathbf{L}_j, 0) + I_{s,j}] + F\, r\, \mathbf{C}_{\text{sky}}(\mathbf{R}) + \mathbf{C}_{\text{caustic}} + \mathbf{E}, \tag{4.16}$$

where $\mathbf{E}$ is the emitted colour of glowing materials. The code is short:

```
// shade() in common.glsl (shortened)
float hemi = N.y * 0.5 + 0.5;
vec3 amb  = mix(uGroundColor, skyAvg, hemi) * uAmbient;
vec3 col  = albedo * amb;                                   // ambient
float ndl = max(dot(N, L), 0.0);                            // Lambert
vec3 H    = normalize(L + V);                               // Blinn half vector
float spec = pow(max(dot(N, H), 0.0), shininess) * ks * (shininess + 8.0) / 32.0;
col += (albedo * ndl + spec) * uSunColor * shadow;          // diffuse + specular
for (int i = 0; i < uNumPoint; i++) { /* same terms, times 1/(1+0.06 d^2) */ }
```

4.4.3 Light sources and atmosphere

There is one daylight for the whole world (`World::lighting`): a warm directional sun, a blue sky, and a soft ambient term. Inside the powerhouse up to four point lights are added. When the camera dives below the reservoir surface the lighting switches to an under-water set-up with a blue-green fog. Water absorbs red light first, so the fog uses a different absorption for each colour channel, following the Beer–Lambert law:

$$\mathbf{T}(d) = \exp(-d\,\rho\,(2.2,\ 0.9,\ 0.65))\,, \qquad \mathbf{C}' = \mathrm{mix}\Big(\mathbf{C}\odot\mathbf{T},\ \mathbf{C}_{\mathrm{fog}},\ 1-e^{-1.2\,\rho d}\Big)\,. \tag{4.17}$$

Above water, ordinary exponential fog $f = 1 - e^{-\rho d}$ softens distant mountains. A moving caustic pattern [14] is added to surfaces under water.

4.4.4 Tone mapping and gamma

The lighting is calculated in linear high dynamic range, where values above one are allowed (the sun on bright concrete, glowing current). Before display the colour is compressed with the filmic ACES curve fitted by Narkowicz [15] and then gamma-corrected for the monitor:

$$\mathrm{ACES}(x) = \frac{x\,(2.51x + 0.03)}{x\,(2.43x + 0.59) + 0.14}, \qquad \mathbf{C}_{\mathrm{out}} = \mathrm{ACES}(e\,\mathbf{C})^{1/2.2}, \tag{4.18}$$

where e is the exposure of the scene.

4.5 Gouraud and Phong Shading

Lighting can be calculated at different places in the pipeline, and this changes the result (Figure 4.1).

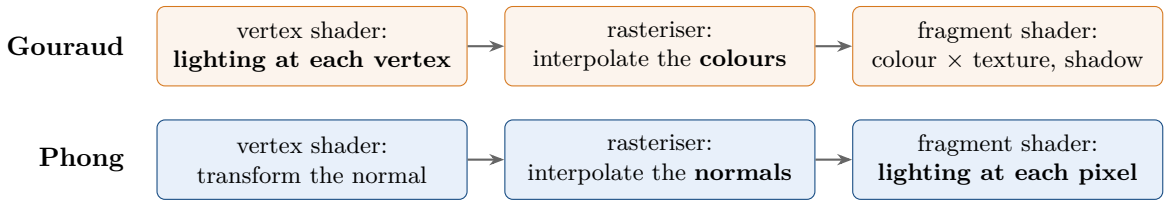


Figure 4.1: Where the lighting is evaluated in Gouraud and Phong shading.

Gouraud shading [5] evaluates the lighting equation only at the three vertices of a triangle and lets the rasteriser blend the resulting colours. For a pixel with barycentric weights $\lambda_0 + \lambda_1 + \lambda_2 = 1$,

$$I(\mathbf{x}) = \lambda_0 I_0 + \lambda_1 I_1 + \lambda_2 I_2. \tag{4.19}$$

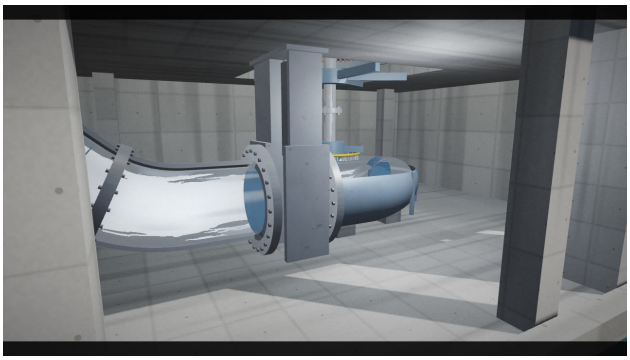
It is cheap, but a highlight that falls between the vertices of a large triangle is lost, because no vertex sees it. In the project, `gouraud.vert` computes the ambient and point-light colour, the sun's diffuse colour and the sun's specular colour per vertex. `gouraud.frag` only multiplies them by the texture and applies the shadow per pixel, so shadows stay sharp while the lighting itself stays per vertex.

Phong shading [3] interpolates the *normal* instead and evaluates the full lighting equation for

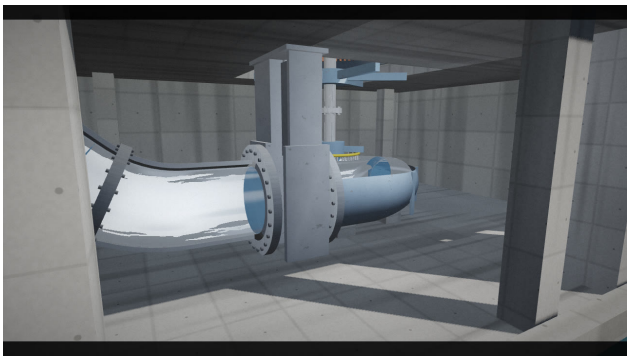
every pixel:

$$\mathbf{N}(\mathbf{x}) = \text{normalize}(\lambda_0\mathbf{N}_0 + \lambda_1\mathbf{N}_1 + \lambda_2\mathbf{N}_2)\,, \qquad I(\mathbf{x}) = \text{shade}(\mathbf{N}(\mathbf{x}),\, \mathbf{x}). \tag{4.20}$$

Highlights are sharp and correctly placed even on big triangles. The project implements this in `lit.vert` and `lit.frag`. Both paths call the same lighting terms from `common.glsl`, so the comparison is fair: the only difference is *where* the equation is evaluated.



(a) Phong: the sun's highlight on the large ceiling panels and the bright reflection on the pipe are clear.



(b) Gouraud: the same view; the highlight between the vertices of the ceiling is lost.

Figure 4.2: The same frame of scene 6 rendered with Phong and with Gouraud shading.

G cycles through three modes: *Mixed* (the default: the large, smooth terrain uses Gouraud, everything else uses Phong), *all Phong* and *all Gouraud*. The Renderer picks the shader program for each draw call according to the mode and a per-material preference.

4.6 Shadow Mapping

Shadows use the two-pass shadow-map method [7]. In the first pass the scene is drawn from the sun's point of view into a 2048×2048 depth texture. The light's view matrix looks from $\mathbf{c} + 2r\,\mathbf{L}$ towards the centre $\mathbf{c}$ of the area of interest, and an orthographic projection covers a square of half-size r. Each scene chooses $\mathbf{c}$ and r, so the resolution is spent where the camera is looking. In the second pass, every pixel is transformed into the same light space:

$$\mathbf{p}_L = \mathbf{P}_L\,\mathbf{V}_L\,(\mathbf{x} + \delta\,\mathbf{N}), \qquad (u, v, z) = \tfrac{1}{2}\,\mathbf{p}_L.xyz/\mathbf{p}_L.w + \tfrac{1}{2}. \tag{4.21}$$

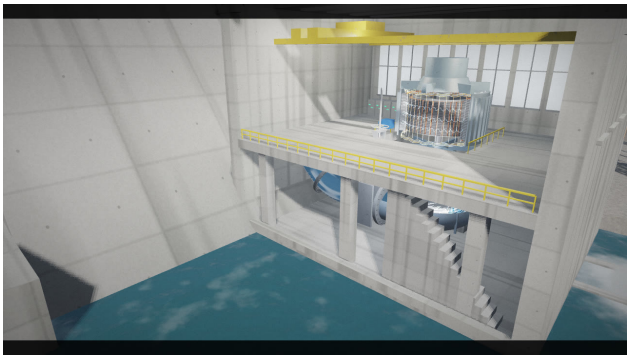
The point is in shadow if something nearer to the sun was stored in the depth map at (u, v). Moving the point a little along its normal (δ, three texels) and a small depth bias b prevent "shadow acne", the stripes that appear when a surface shadows itself. Polygon offset is also enabled in the shadow pass. Hard shadow edges are softened with 3×3 percentage-closer filtering [8]:

$$S = \frac{1}{9}\sum_{i=-1}^{1}\sum_{j=-1}^{1}\Big[\,z - b \le D(u + 1.25\,i\,\Delta,\;\; v + 1.25\,j\,\Delta)\Big], \tag{4.22}$$

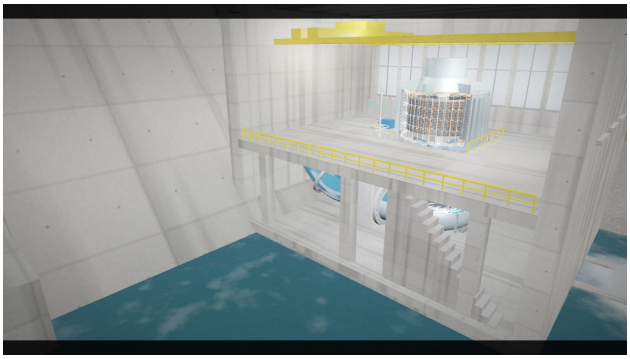
where D is the stored depth, Δ the size of one texel and $[\cdot]$ is one when the condition holds and zero otherwise.

4.7 Procedural Textures

Every surface pattern is calculated in the fragment shader (`materials.glsl`), so no image files are needed. The basis is *value noise*: random values at the corners of a grid, blended with the smooth



(a) Shadows on: the dam, piers and railings cast soft shadows.



(b) Shadows off (L): the scene looks flat.

Figure 4.3: Effect of shadow mapping on the preview scene.

curve $s(t) = 3t^2 - 2t^3$. Several octaves are added to form fractal noise,

$$\mathrm{fbm}(\mathbf{p}) = \sum_{i=0}^{3} \frac{1}{2^{i+1}} \,\mathrm{noise}\left(2^i \mathbf{p}\right), \tag{4.23}$$

which gives detail at several scales, like real stone. The random values come from Hoskins' arithmetic hash [13], which behaves the same on every GPU. Table 4.2 lists the 15 patterns.

Table 4.2: Procedural material patterns.

Pattern	Idea	Pattern	Idea
Concrete	formwork panels, tie holes, stains	Copper	warm metal with wire lines
Steel	brushed noise, high specular	Hologram	glowing grid lines
Paint	smooth colour with slight wear	Leaf	vein pattern, alpha cut-out
Bark	stretched noise along the log	Light beam	soft additive shafts under water
Terrain	rock, grass and silt by height and slope	Rock	strong fbm, dark cracks
Soil	dark wet noise	Plastic	shiny, coloured
Tiles	floor tiles with grout lines	Flow	moving glowing pulses
Grating	wire mesh cut out with `discard`		

The terrain is a good example of combining rules. Its colour is chosen from slope and height: steep parts become rock, flat parts above the water become grass, and parts below the water line become wet silt, with fbm noise breaking up the borders.

4.8 Water Rendering

Water appears in many forms: the reservoir and river, thin sheets on the spillway, the jet in the conduit, the tube of water in the penstock, the swirl in the spiral casing, and the cut faces of water in the cut-away views. One shader pair, `water.vert` and `water.frag`, handles all of them through a *mode*.

Waves. Vertices on a free surface are lifted by a sum of three sine waves that drift with the

current u:

$$h(x,z,t) = A\big[0.5\sin(fq_x + 1.3t) + 0.3\sin(1.4fq_z - 1.1t + 0.5fq_x) \\ + 0.2\sin(2.3f(q_x+q_z) + 2.1t)\big], \qquad \mathbf{q} = (x - ut,\ z). \quad (4.24)$$

In the fragment shader the same function is differentiated numerically to get a detailed normal, $\mathbf{N} = \text{normalize}(\mathbf{N}_0 - \nabla h)$, which is a form of bump mapping.

Reflection and colour. The final colour mixes the water's body colour with the reflected sky according to the Fresnel term (Schlick, $F_0 = 0.02$ for water), plus a sharp sun glitter $\max(\mathbf{N}\cdot\mathbf{H}, 0)^{900}$ and foam where the water is fast.

Snell's window. When the camera is under water and looks up, the shader refracts the view ray out of the water with $n = 1.33$. Inside a cone of about $48.8°$ (the critical angle, $\arcsin(1/1.33)$) the sky is visible; outside it the ray is totally reflected and the surface looks like a mirror, which is what divers see.

Moving water fronts. The water in the conduit and penstock has a front that moves with the simulation. Fragments beyond the front are discarded, and in the penstock the vertex shader pulls the vertices just beyond the front towards the pipe axis using $r' = r\sqrt{1-k^2}$, which gives the advancing water a rounded nose.

4.9 Ray Tracing

Scene 5 is drawn by a completely different method. Instead of rasterising triangles, one full-screen triangle is drawn and the fragment shader `raytrace.frag` traces a ray for every pixel through a scene described by exact mathematics: a glass penstock with steel sleeves, flanges, concrete supports, a glossy floor and a back wall. The algorithm is Whitted-style recursive ray tracing [6], unrolled to a fixed depth because GLSL does not allow recursion. Figure 4.4 shows the rays involved.

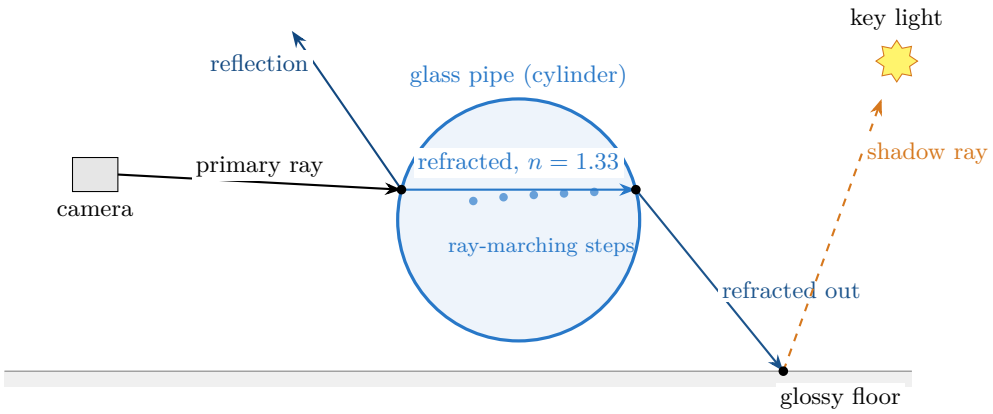


Figure 4.4: Rays traced for one pixel that hits the glass pipe.

4.9.1 Primary rays

For a pixel at normalised device coordinates $(x_n, y_n) \in [-1,1]^2$, the ray starts at the camera $\mathbf{o}$ and points along

$$\mathbf{d} = \text{normalize}\big(\mathbf{F} + x_n \tan\tfrac{\varphi}{2}\, a\, \mathbf{R} + y_n \tan\tfrac{\varphi}{2}\, \mathbf{U}\big)\,, \quad (4.25)$$

where $\mathbf{F}$, $\mathbf{R}$, $\mathbf{U}$ are the camera's forward, right and up vectors. The ray is $\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}$, and the task is to find the smallest positive t at which it hits something.

4.9.2 Ray–surface intersections

Infinite cylinder (the pipe). The pipe has axis point $\mathbf{A}$, unit direction $\mathbf{D}$ and radius r. Removing the components along the axis, $\mathbf{d}_\perp = \mathbf{d} - (\mathbf{d}\cdot\mathbf{D})\mathbf{D}$ and $\mathbf{o}_\perp = (\mathbf{o}-\mathbf{A}) - ((\mathbf{o}-\mathbf{A})\cdot\mathbf{D})\mathbf{D}$, the hit

condition $\|\mathbf{o}_\perp + t\,\mathbf{d}_\perp\| = r$ becomes the quadratic

$$a\,t^2 + 2b\,t + c = 0, \qquad a = \mathbf{d}_\perp \cdot \mathbf{d}_\perp, \quad b = \mathbf{o}_\perp \cdot \mathbf{d}_\perp, \quad c = \mathbf{o}_\perp \cdot \mathbf{o}_\perp - r^2, \tag{4.26}$$

$$t_{1,2} = \frac{-b \mp \sqrt{b^2 - ac}}{a}. \tag{4.27}$$

There is no hit when $b^2 - ac < 0$. The normal at the hit point $\mathbf{q}$ is the radial direction, normalize$((\mathbf{q} - \mathbf{A}) - ((\mathbf{q} - \mathbf{A}) \cdot \mathbf{D})\mathbf{D})$. The axial position s decides the material: the pipe repeats every 16 m, and the first 11 m of each period are glass, the rest steel.

```
vec2 cylinder(vec3 ro, vec3 rd, float r) {        // raytrace.frag
    vec3 oc  = ro - uPipeA;
    vec3 rdp = rd - uPipeDir * dot(rd, uPipeDir);  // remove the axial parts
    vec3 ocp = oc - uPipeDir * dot(oc, uPipeDir);
    float a = dot(rdp, rdp), b = dot(ocp, rdp), c = dot(ocp, ocp) - r * r;
    float h = b * b - a * c;                        // discriminant
    if (h < 0.0) return vec2(-1.0);                 // miss
    h = sqrt(h);
    return vec2((-b - h) / a, (-b + h) / a);        // entry and exit
}
```

Capped cylinders (flanges and bands) use Quilez's closed-form intersection [12], which tests the side and both end caps in one function.

Boxes (supports) use the slab method: for each axis the ray enters and leaves the slab between two planes at $t_0 = (\mathbf{b}_{\min} - \mathbf{o})/\mathbf{d}$ and $t_1 = (\mathbf{b}_{\max} - \mathbf{o})/\mathbf{d}$ (component-wise). The box is hit if $t_{\text{near}} = \max(\min(t_0, t_1)) \le t_{\text{far}} = \min(\max(t_0, t_1))$ and $t_{\text{far}} > 0$.

Planes (floor and wall): for the floor $y = 0$, $t = -o_y/d_y$ when $d_y < 0$.

4.9.3 Shading, shadows, reflection and refraction

Shadow rays. From a hit point a second ray is sent towards the key light. If it hits steel or concrete, the point is in shadow (only 12 % of the light remains). If it hits glass, blue-tinted light passes through, so the floor under the pipe receives a coloured shadow.

Reflection. The glossy floor, the glass and the steel send a reflection ray in the mirror direction

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{N})\,\mathbf{N}, \tag{4.28}$$

weighted by the Fresnel term. For glass, $F = 0.04 + 0.96\,(1 - \cos\theta_i)^5$.

Refraction. At the glass the ray bends into the water according to Snell's law $n_1 \sin\theta_1 = n_2 \sin\theta_2$. In vector form, with $\eta = n_1/n_2 = 1/1.33$ and $\cos\theta_i = -\mathbf{N} \cdot \mathbf{d}$,

$$\mathbf{t} = \eta\,\mathbf{d} + \left(\eta \cos\theta_i - \sqrt{1 - \eta^2(1 - \cos^2\theta_i)}\right)\mathbf{N}. \tag{4.29}$$

When the ray leaves the water the ratio is inverted. If the value under the square root is negative, the light cannot leave the water (total internal reflection), and the ray is reflected instead.

Volume ray marching. The water does not have a single surface; it is a volume full of glowing streaks. Between the entry and exit points the ray is split into 36 steps. At each step the shader evaluates a density $\rho(\mathbf{p})$ built from noise that moves along the pipe at the flow speed, adds the emitted light and reduces the transmittance T with the Beer–Lambert law:

$$L \mathrel{+}= \mathbf{e}(\mathbf{p})\,\rho(\mathbf{p})\,\Delta t\,T, \qquad T \leftarrow T\,\exp(-\Delta t\,(0.05 + 0.12\,\rho(\mathbf{p}))). \tag{4.30}$$

The start of each march is jittered by a random fraction of a step per pixel, which hides banding. The glass colour is finally $\mathbf{C} = \text{mix}(\mathbf{C}_{\text{transmitted}}, \mathbf{C}_{\text{reflected}}, F)$.

4.9.4 Interaction with the ray tracer

UP and DOWN change the head H in steps of 10 m. The C++ side computes the jet speed $v = \sqrt{2gH}$, the flow $Q = 180\sqrt{H/53}$ and the power from Equation (3.1), and passes the speed to the shader, so the glowing streaks visibly race faster or slower. A moving scan ring and the hologram floor grid give the scene a futuristic, diagram-like look.

4.10 Animation and Physics

4.10.1 Time step and speed control

All motion is driven by the frame time. Every scene update receives $\Delta t_{\text{scene}} = \Delta t \times s$ with $s \in \{0.25, 0.5, 1, 2\}$, so the whole world (water, machines, particles, cameras) can be watched in slow or fast motion. The water is additionally shown at 0.55–0.6 of its real speed so the eye can follow it.

4.10.2 Free-falling water

The spillway sheets and the penstock water are modelled as fronts that travel along a path. The speed at a point depends on how far the water has dropped, from energy conservation:

$$v(s) = \sqrt{v_0^2 + 2\,g\,\Delta h(s)}, \qquad s \leftarrow s + v(s)\,\Delta t, \tag{4.31}$$

with $v_0 = 2$ m/s on the spillway and 6 m/s in the penstock. The water therefore starts slowly and accelerates downhill. When a gate closes, the tail of the water runs down the same way, so the chute drains instead of the water simply vanishing.

Under the intake gate the water jets out with Torricelli's speed [19],

$$v_{\text{jet}} = \sqrt{2\,g\,h} = \sqrt{2 \times 9.81 \times 17} \approx 18.3 \text{ m/s}, \tag{4.32}$$

where h is the depth of the opening below the reservoir surface. The conduit's water surface is rebuilt every frame as a height profile with a bulge behind the gate, a rounded front and a rising level.

4.10.3 Machines with inertia

Real machines do not jump to a new state. Gates, the valve, the guide vanes, the field current and the breaker move at limited rates:

$$x \leftarrow x + \text{clamp}\big(x_{\text{target}} - x,\ -r\,\Delta t,\ r\,\Delta t\big). \tag{4.33}$$

The runner speed follows the flow like a first-order system with a time constant of about two seconds:

$$\omega \leftarrow \omega + \big(\omega_{\text{target}} - \omega\big)\ \min(0.45\,\Delta t, 1), \qquad \omega_{\text{target}} = \Omega_v \min(1, 1.25\,q). \tag{4.34}$$

This is why the turbine keeps turning for a short time after the water stops.

4.10.4 Floating debris

Each piece of trash is simulated in the local flow frame. The current drags it along x, a spring–damper holds it at its floating depth, and it bobs gently:

$$v_x \leftarrow v_x + (u - v_x)\min(1.5\,\Delta t, 1), \tag{4.35}$$

$$y_{\text{target}} = y_{\text{float}} + 0.12\sin(1.6\,t + \phi), \tag{4.36}$$

$$v_y \leftarrow v_y + \big(2.5\,(y_{\text{target}} - y) - 1.8\,v_y\big)\Delta t, \tag{4.37}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\,\Delta t. \tag{4.38}$$

At the trash rack a piece larger than the 0.56 m gap stops, throws a splash of particles and settles against the bars; small leaves squeeze through and are drawn down into the intake.

4.10.5 Particle systems

Spray, mist, outflow foam, splashes, gate spray and under-water dust are particle systems [11]. Each particle has a position, velocity, size, colour and lifetime, and is integrated with explicit Euler steps including gravity and drag:

$$v_y \leftarrow v_y + g\,\Delta t, \qquad \mathbf{v} \leftarrow \frac{\mathbf{v}}{1 + k_d\,\Delta t}, \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\,\Delta t, \qquad \text{size} \leftarrow \text{size} + \gamma\,\Delta t. \tag{4.39}$$

The opacity fades in quickly and out slowly, $\alpha = \alpha_0 \min(1, 8k)(1 - k)$ with $k = \text{age}/\text{lifetime}$. Each particle is drawn as a quad that always faces the camera (a billboard), built from the camera's right and up vectors $\mathbf{x} \pm s\mathbf{R} \pm s\mathbf{U}$. Dead particles are removed by swapping them with the last element, which costs constant time.

4.10.6 Glowing flow lines

The invisible things in the plant (the paths of the water through the turbine, the magnetic field and the electric current) are shown with *flow lines*: thin tubes along a path whose texture coordinate s is the distance along the path. The shader draws bright pulses that move along the line:

$$\phi = \operatorname{fract}(\sigma s - \sigma v t), \qquad \text{pulse} = \operatorname{smoothstep}(0, 0.08, \phi)\,(1 - \operatorname{smoothstep}(0.08, 0.6, \phi)), \tag{4.40}$$

where σ is the number of pulses per metre and v their speed. A *front* value hides everything beyond the point the water or the energy has reached, and the tip of the front glows brighter. The same technique shows 24 water streamlines through the turbine, the field lines around the rotor poles, the orange three-phase current in the stator coils and bus ducts, and the energy travelling 640 m along the power line.

4.10.7 Three-phase oscilloscope

The generator scene draws three sine waves, each shifted by 120°:

$$y_p(u) = A\,\sin\!\left(2\pi(c\,u - k\,t) - \frac{2\pi p}{3}\right), \qquad p = 0, 1, 2, \tag{4.41}$$

where the amplitude A is proportional to the terminal voltage. The waves therefore grow from nothing when the rotor is excited.

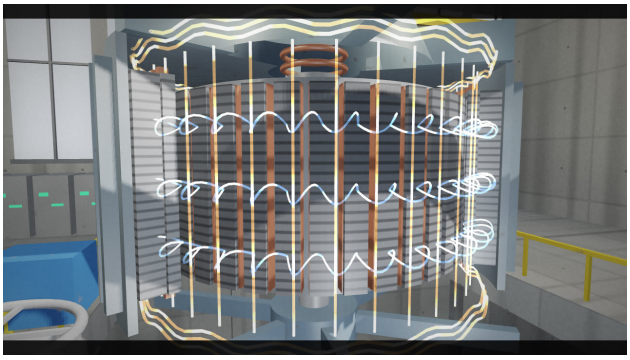
4.11 Post-Processing and Transitions

The 3D image is first rendered into a texture. Three small shaders then give it a cinematic finish (`render/PostFX.cpp`):

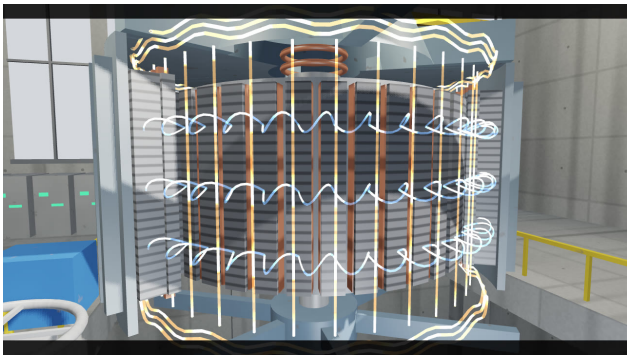
1. **Bright pass** at a quarter of the screen size keeps only the bright parts, using the luminance $Y = 0.2126R + 0.7152G + 0.0722B$ and a soft threshold $k = \operatorname{smoothstep}(0.78, 1, Y)$.
2. **Gaussian blur**, done separably: first horizontally, then vertically. A 9-tap filter with weights $w = (0.227, 0.195, 0.122, 0.054, 0.016)$ is applied in each direction [16]. Two one-dimensional passes cost 2×9 samples instead of 81.
3. **Composite**: the blurred glow $\mathbf{g}$ is added to the image (*bloom*), the corners are darkened with $1 - 0.76\,\|\mathbf{d}\|^2$ (*vignette*), and a little animated noise is added (*film grain*).

The whole pass costs about one millisecond and can be switched off with V (Figure 4.5).

Cross-dissolve. When the render style changes completely, for instance from the raster world to the ray tracer, the last frame is kept in a texture (`render/FrameCapture.cpp`) and blended over the new scene with an opacity that falls from one to zero over 1.1 s along the smooth curve $\alpha = x^2(3 - 2x)$. Elsewhere the camera simply flies on.



(a) Bloom on: the current and field lines glow.



(b) Bloom off (`V`).

Figure 4.5: Post-processing on the generator scene.

4.12 User Interface and Text

The HUD (`ui/Ui.cpp`) is drawn in pixel coordinates with an orthographic mapping: a pixel (x, y) goes to $(2x/W - 1,\ 1 - 2y/H)$ in clip space. Text uses a hand-made font (`ui/Font.cpp`): each of the 63 glyphs (capital letters, digits and punctuation) is written in the source as seven strings of five characters, where `#` marks a lit pixel. At start-up the glyphs are packed into a small texture with 6×8 cells and sampled with nearest filtering, so the letters stay crisp at any integer scale. Text positions are snapped to whole pixels so that a glyph never samples its neighbour in the atlas.

Labels that point at 3D parts use the same projection as the renderer: the part's world position is multiplied by **PV**, divided by w, and mapped to the screen. A leader line then joins the point to the label, and labels are pushed out of the title and telemetry panels if they would overlap them.

4.13 Interaction

Interaction happens on three levels. **Scene buttons** at the bottom of the screen drive the story; each has a keyboard shortcut shown under it. A button whose cause has not happened yet is drawn in grey with a hint ("open the gate first", "wait, the casing is filling"), which teaches the order of operation without any extra text. **Global controls** work in every scene (Table 4.3). **The mouse** orbits, pans and zooms the camera on top of the director's camera.

Table 4.3: Global controls.

Input	Action	Input	Action
Left mouse drag	Orbit around the target	`L`	Shadows on and off
Right drag, `W` `A` `S` `D` `Q` `E`	Move (pan) the camera	`V`	Bloom, vignette and grain on and off
Mouse wheel	Zoom	`H`	Hide or show the HUD
`C`	Back to the director's camera	`P`	Automatic play-through of the whole story (about three minutes)
`[` `]` or buttons	Speed 0.25×, 0.5×, 1×, 2×	`1`–`9`, `0`	Jump to scenes 1–10
`G`	Shading: Mixed, Phong, Gouraud	`N`/`B`, arrows	Next or previous scene
`F12`	Save a screenshot as a BMP file	`R`	Restart the current scene
		`Esc`	Quit

Chapter 5

Objects of the Scene

Each object has its own source file in `src/objects/` and is built entirely from the primitives of Section 4.1. The world uses metres, with x along the dam crest, y up and z downstream. Table 5.1 gives an overview, and the sections that follow describe how each object is built and how it moves.

Table 5.1: Objects in the scene.

Object	File	Built from	Moving parts
Valley terrain	`Terrain.cpp`	260×260 height-field grid from fbm noise	–
Boulders	`Rocks.cpp`	spheres displaced by noise	–
Gravity dam	`Dam.cpp`	extruded profile, piers, crest, gantry crane	3 spillway gates, cut block
Spillway water	`SpillwayWater.cpp`	sheets following the chute profile	free-falling fronts
Intake structure	`IntakeStructure.cpp`	boxes, deck, rake motors	–
Trash rack	`TrashRack.cpp`	flat bars, beams, brackets	–
Floating debris	`Debris.cpp`	branch, logs, plastic, straw, leaves	29 pieces per wave
Intake gate	`IntakeGate.cpp`	leaf, ribs, cylinders, rods	gate and 2 rods
Penstock	`CutawayPipe.cpp`	swept tube with cut, rings, bolts	water front
Powerhouse	`Powerhouse.cpp`	walls, floors, windows, railings, crane	shaft
Main inlet valve	`MainInletValve.cpp`	flanges, body, bonnet, gate, wheel	gate, stem, wheel
Francis turbine	`TurbineUnit.cpp`	spiral sweep, vanes, runner, draft tube	24 vanes, ring, runner
Generator	`Generator.cpp`	stator ring, 72 slots, rotor, 40 poles	rotor
Transformer	`Transformer.cpp`	tank, radiators, fans, bushings, ducts	4 fans
Switchyard	`Switchyard.cpp`	gantries, breakers, CTs, arresters	status lamps
Power line	`TransmissionLine.cpp`	10 lattice towers, sagging wires	energy glow
Flow lines	`FlowLines.cpp`	thin tubes along paths	moving pulses
Particles	`ParticleSystem.cpp`	camera-facing quads	6 systems

5.1 Terrain, Reservoir and River

The valley is a height function $h(x, z)$ evaluated on the CPU at 260×260 points. It combines a river channel, the reservoir bed upstream, mountain sides that rise away from the river and fbm noise for natural irregularity. Normals are taken from the gradient of h. The terrain is the largest and smoothest object, so in Mixed mode it uses Gouraud shading, which is a sensible use of the cheaper method. The reservoir (level 56 m) and the river (level 3 m) are large water grids, and boulders are spheres pushed in and out by noise with a flattened bottom.

5.2 Dam and Spillway

The concrete gravity dam is 300 m long with its crest at 63 m. Its cross-section is a 2D profile, vertical on the upstream face and sloping on the downstream face, extruded along the crest. In the middle, three 11 m spillway bays separated by 4 m piers carry vertical-lift gates that translate up by 9.5 m. The power waterway passes through the dam at $x = 58$ m: an intake mouth in the upstream face, an 11×10 m conduit, a vertical slot for the intake gate and a bulkhead where the penstock starts. The conduit itself is not a hole cut out of a solid; it is the gap left between four extruded solid pieces. The section between $x = 24.5$ and 52.5 m is a separate "cut block" that lifts away for the cut-away views.

5.3 Intake Structure, Trash Rack and Debris

In front of the intake a concrete structure with side piers and a top deck holds a cage of trash-rack panels. Each panel is a row of flat steel bars 0.56 m apart between a top and a bottom beam. The near half of the structure forms a separate group that is hidden in the cut-away view so the cage can be seen from the side. Debris pieces are small meshes (a forked branch, logs, crumpled plastic, straw and leaves) drawn with bark, plastic and leaf patterns; one wave contains 29 pieces, and `T` sends another.

5.4 Intake Gate and Penstock

The intake gate is a steel leaf with ribs, lifted by two hydraulic cylinders standing on the dam crest. The penstock is a 4 m diameter steel pipe about 58 m long that drops 33 m from the bulkhead to the powerhouse. It is a circle swept along a path with rounded bends; only part of its circumference is kept, so it is open on the camera side. Stiffener rings with bolts go round the full pipe. The removed part can be shown as a glowing X-ray grid, and the water inside is a separate tube whose front moves.

5.5 Powerhouse

The powerhouse stands at the foot of the dam and is drawn as a cut-away with its front wall removed. It has a turbine pit, the main shaft with a guide bearing and a bolted coupling flange, a generator floor at 14 m with yellow railings and control cabinets, large windows, lamps (point lights) and an overhead crane. The penstock ends at the main inlet valve inside the pit.

5.6 Main Inlet Valve

The valve sits between the penstock and the spiral casing. Two bolted flanges hold a cast body with a tall bonnet. Inside, a heavy gate slides up by 4.4 m into the bonnet when the valve opens, and a hand-wheel on top turns only while the gate is moving. The body is cut open on the camera side so the gate can be seen.

5.7 Francis Turbine

The turbine (`TurbineUnit.cpp`) is the most detailed object. Water enters a *spiral casing*: a tube swept along a spiral whose radius shrinks as it wraps round, so that the water keeps the same speed as it is fed inwards. Inside the casing, 12 fixed *stay vanes* hold the structure together, and 24 movable

guide vanes control how much water reaches the runner. The guide vanes turn together through an operating ring. The *runner* has 15 curved blades built as twisted strips; the water turns inwards through them and then flows down the *draft tube* cone to the river. The casing is open on top, and in section view the near half is hidden. For that view a second set of streamlines is used that keeps only the far half, so the runner stays visible.

5.8 Generator

The vertical generator stands on the generator floor above the turbine. The *stator* is a ring of thin steel sheets with 72 slots, each holding a copper coil that loops round at the top and bottom (end windings). The *rotor* has a hub, spider arms, a rim and 40 poles, each wrapped in a copper field coil. The housing has a 120° window cut towards the camera. Exciting the rotor makes blue magnetic field lines appear around the poles; once the rotor spins with a field, orange current pulses run through the stator coils at a rate set by the frequency.

5.9 Bus Ducts and Step-up Transformer

Three isolated-phase bus ducts carry the generator current out through the wall and drop onto the low-voltage bushings of the transformer. The transformer has a green tank, radiator banks, four cooling fans that spin only while there is voltage, a conservator tank and tall brown high-voltage bushings. The X-ray view makes the tank transparent and shows the iron core with three legs, each carrying a low-voltage winding with few turns and a high-voltage winding with many turns. This is the $1:20$ turns ratio that raises 11 kV to 220 kV.

5.10 Switchyard and Power Line

The switchyard follows the order in which the power flows: transformer bushings, a gantry, a disconnector, the circuit breaker, current transformers, surge arresters and the line gantry. Status lamps on the breakers fade from red (open) to green (closed). The power line has ten lattice steel towers marching down the valley. Six conductors in two circuits and an earth wire hang between them; each span is a sagging curve, a parabola close to the true catenary, $y(x) = y_{\mathrm{lin}}(x) - 4\,s\,u(1-u)$ for a span parameter $u \in [0,1]$ and sag s. When the breaker closes, a glow travels along the conductors at 140 m/s and the camera follows it.

5.11 Summary of Moving Objects

Table 5.2 counts everything that moves in the world.

Table 5.2: Moving objects in the world.

Group	Count	Members
Solid parts moved by matrices	44	3 spillway gates, dam block, intake gate, 2 rods, valve gate and stem, hand-wheel, 24 guide vanes, operating ring, runner, shaft, rotor, 4 fans, 2 scan rings
Floating debris	29 per wave	branch, logs, plastic, straw, leaves
Moving water bodies	7	spillway, conduit, penstock, spiral casing, reservoir waves, river, ray-traced streams
Glowing flow animations	6	turbine streamlines, magnetic field, stator current, bus ducts, switchyard, power line
Particle systems	6	spray, mist, outflow foam, splashes, gate spray, underwater dust

Chapter 6

Scene-by-Scene Walkthrough

This chapter follows the program from start to finish. All pictures were rendered by the program itself at 1280×720 with the HUD visible, so the buttons, telemetry and labels can be seen.

6.1 Scene 1: Preview of the Finished Plant

The program opens on the running plant (Figure 6.1). The front of the powerhouse is cut away to show the generator on its floor with the turbine below it, water pours down the spillway, and the telemetry shows 150 rpm and 86 MW. Stop water closes the spillway and the intake; the spillway drains with real free-fall physics and the turbine runs down. Show water movement starts the journey.



Figure 6.1: Scene 1: the running plant with the cut-away powerhouse.

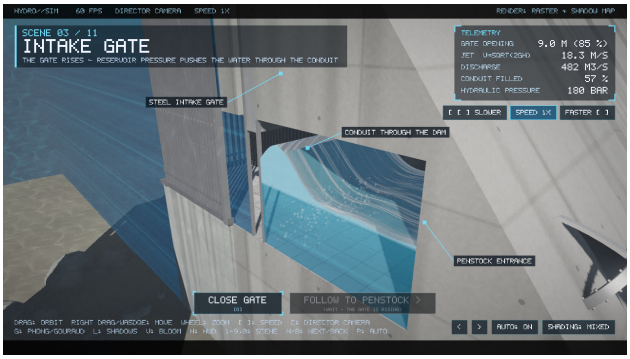
6.2 Scenes 2 and 3: Reservoir, Trash Rack and Intake Gate

In scene 2 the camera flies over the dam crest and dives under the reservoir surface, where the lighting changes to blue under-water light with caustics and drifting dust. A dissolve leads to a cut-away of the trash rack. Branches, logs and plastic drift in with the current and stop at the bars, while small leaves slip through (Figure 6.2a). The telemetry counts the trapped and passed pieces.

In scene 3 the dam block lifts away to reveal the intake gate. When it opens, the jet under the gate shoots into the conduit at about 18 m/s and the conduit fills. The button to follow the water stays grey until the gate is fully open (Figure 6.2b).



(a) Trash stopped at the rack.

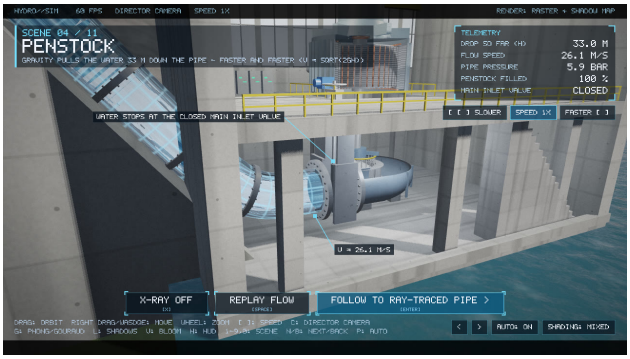


(b) The intake gate rising; water fills the conduit.

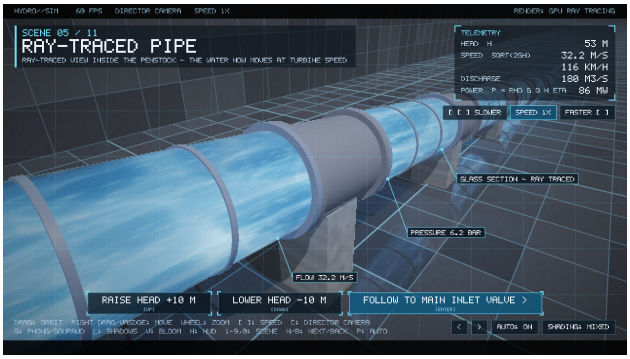
Figure 6.2: Scenes 2 and 3.

6.3 Scenes 4 and 5: Penstock and Ray-Traced Pipe

The water rushes down the open penstock and accelerates as it drops (Figure 6.3a). Half way down the pipe switches to a glowing X-ray grid, and at the bottom the water stops at the closed main inlet valve. A dissolve then leads into the ray-traced scene (Figure 6.3b), where the glass pipe reflects in the floor, casts a blue shadow and shows glowing streams inside.



(a) Water rushing down the penstock.

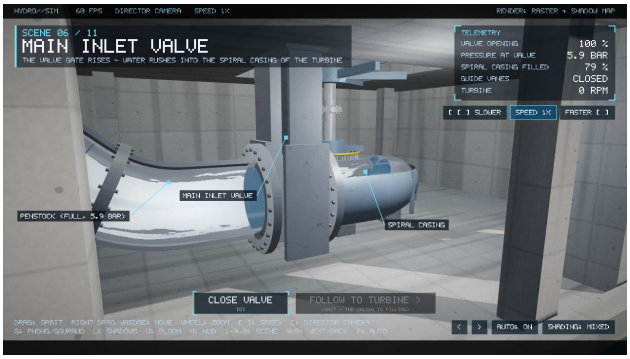


(b) The ray-traced penstock.

Figure 6.3: Scenes 4 and 5.

6.4 Scenes 6 and 7: Main Inlet Valve and Francis Turbine

Opening the valve turns the hand-wheel and lifts the gate, and the water moves on into the spiral casing (Figure 6.4a). In scene 7 the camera looks down into the open casing: blue streamlines swirl round it, pass between the stay and guide vanes and enter the runner (Figure 6.4b). Opening the guide vanes lets the water in, and the runner speeds up with inertia towards 150 rpm while the telemetry shows flow, torque and mechanical power. The section view (Figure 6.5) shows the water turning down through the runner into the draft tube.



(a) The main inlet valve; the casing is filling.



(b) Top view of the spiral casing and vanes.

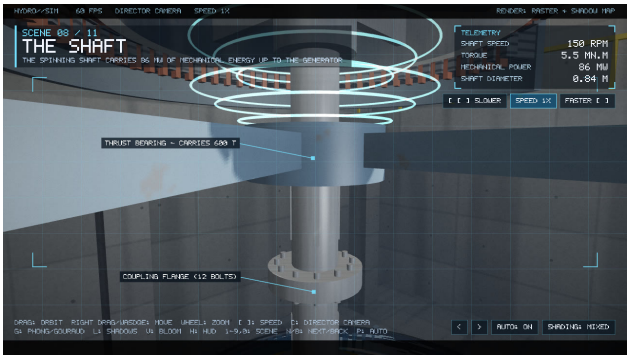
Figure 6.4: Scenes 6 and 7.



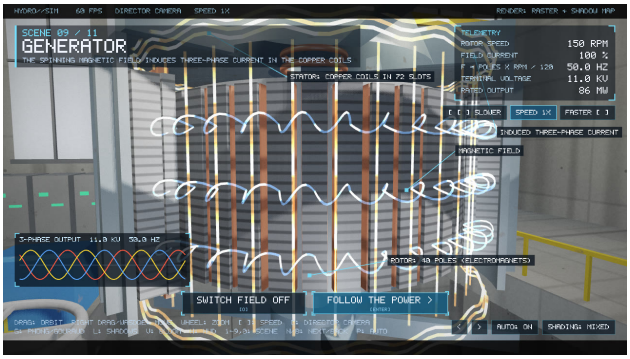
Figure 6.5: Section view of the Francis turbine with the streamlines leaving through the draft tube.

6.5 Scenes 8 and 9: Shaft and Generator

Scene 8 is an automatic hologram-style rise along the turning shaft, past the guide bearing and the coupling flange, to the generator. In scene 9 (Figure 6.6) the viewer excites the rotor. The field current rises, magnetic field lines appear, orange current pulses run through the stator coils, and the oscilloscope shows the three phases growing to 11 kV at 50 Hz.



(a) Scene 8: the shaft and coupling flange.



(b) Scene 9: the excited generator and oscilloscope.

Figure 6.6: Scenes 8 and 9.

6.6 Scenes 10 and 11: Transformer and Grid

The camera flies out over the powerhouse roof to the transformer yard. The bus ducts glow with current, and the X-ray view shows the windings inside the tank (Figure 6.7a). In the switchyard

the disconnector, breaker, current transformers and surge arresters are labelled. Closing the breaker sends the energy along the power line, and the camera flies with it to the horizon (Figure 6.7b). At the end, a summary of the whole energy chain appears (Figure 6.8), and Replay journey returns to the preview.



(a) Scene 10: transformer in X-ray view.



(b) Scene 11: energy travelling along the line.

Figure 6.7: Scenes 10 and 11.

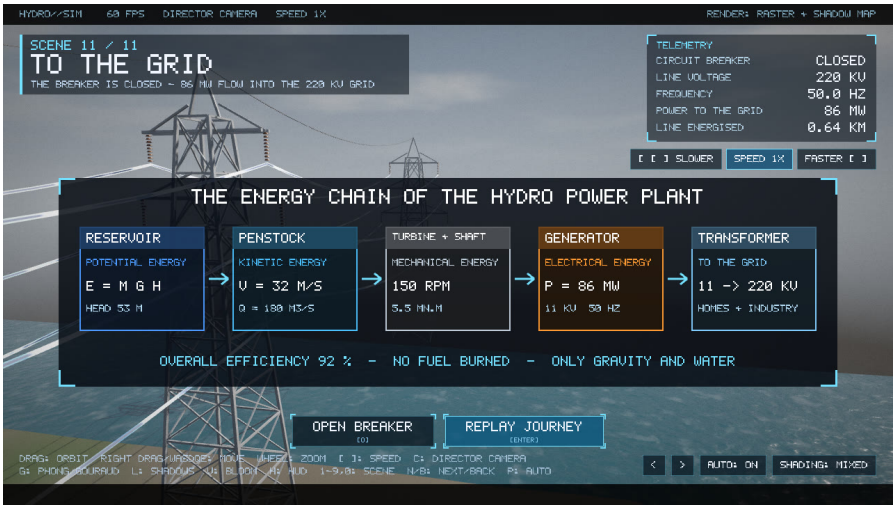


Figure 6.8: The energy-chain summary at the end of the journey.

Chapter 7

Results, Testing and Discussion

7.1 Main Results

The final program meets all the goals set in Chapter 1. It presents the whole plant, from reservoir to grid, as one continuous, interactive world, demonstrates all six compulsory topics (Table 2.1), and implements every object and feature of the proposal in the context of a hydroelectric plant (Tables 2.2 and 2.3). The plant is built from 18 object files and runs in 11 connected scenes, and apart from GLFW and GLAD every piece of maths, geometry and texture is the project's own code.

7.2 Testing Method

A graphics program has to be checked by looking at it, so the program includes tools that make looking repeatable:

- **Shot mode** (`--shot <scene> <seconds> <file>`) starts a scene, runs the story for a fixed simulated time at a fixed time step, and saves a picture. Button presses can be scripted with `--press`. Every figure in this report was made this way, which also means any figure can be reproduced exactly.
- **Automatic test** (`--autotest`) runs the real frame loop on AUTO, including the dissolves. The full chain reaches the energy summary at about 180 seconds.
- **Benchmark** (`--bench`) measures the real frame time of every scene.
- Switches `--shading`, `--noshadows` and `--nopost` produce the matched comparison images of Chapter 4.

7.3 Performance

The plant contains far more detail than any one view shows, so the world culls what cannot be seen. The detailed machines (valve, turbine, generator) are drawn only when the camera is within 160 m of the powerhouse, and the transformer, switchyard and power line are skipped while the camera is inside the hall. Meshes are built once and reused: one guide-vane mesh is drawn 24 times and one spillway-gate mesh three times. Bloom works at a quarter of the screen size, and particles live in a fixed-capacity array with constant-time removal.

Table 7.1 gives the measured frame rates at 1600×900 on a laptop with an NVIDIA GeForce GTX 1650. Every scene stays above the 30 frames per second usually needed for smooth motion. The ray-traced scene is the most expensive, because each pixel traces several rays and marches 36 steps through the water.

7.4 Limitations

- The interiors are cut-away teaching diagrams, not fully closed engineering models.
- The physics is simplified for teaching. Water fronts follow free-fall and Torricelli formulas rather than a fluid simulation, and the water is shown slower than real speed so the eye can follow it.
- The plant has a single turbine–generator unit.
- Ray tracing is used for the penstock scene only; the rest of the world is rasterised. Shadow rays in the ray tracer produce hard shadows.
- Only the sun casts shadows; the point lights in the powerhouse do not.
- There is no sound.

Table 7.1: Measured performance at 1600×900 (GTX 1650).

#	Scene	ms / frame	fps
1	Preview	26.6	37.5
2	Reservoir and trash rack	18.7	53.6
3	Intake gate	20.2	49.4
4	Penstock	20.6	48.6
5	Ray-traced pipe	29.3	34.1
6	Main inlet valve	23.7	42.2
7	Francis turbine	22.8	43.8
8	Shaft	22.7	44.0
9	Generator	27.0	37.0
10	Transformer	18.8	53.1
11	Switchyard and grid	16.6	60.4

Chapter 8

Conclusion and Future Work

8.1 Conclusion

What started as a small sluice-gate scene became a complete hydroelectric power plant that can be explored from the reservoir to the power line. The project uses plain OpenGL and C++, with every shape, texture and matrix written from scratch. It demonstrates lighting with ambient, diffuse and specular terms, every kind of transformation, Gouraud and Phong shading side by side, continuous physics-based animation, rich interaction, and a real ray tracer. On top of these it adds shadow mapping, water rendering, particles, glowing flow lines and a post-processing pass that give the scenes a cinematic look.

The part I am most satisfied with is that the program explains something real. The numbers on screen come from real formulas and agree with each other, the machines only move when their cause allows it, and the viewer learns the order of operation of a power plant by operating it.

8.2 Future Work

- Several turbine units with a control room that shares load between them.
- A simple fluid simulation for the spillway and the reservoir surface.
- Ray-traced reflections and soft shadows for the whole world, not only one scene.
- Shadows from the powerhouse lamps using cube shadow maps.
- Sound: rushing water, the hum of the generator, the click of the breaker.
- A day–night cycle with lights coming on along the transmission line.

References

[1] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 2nd ed. Addison-Wesley, 1996.

[2] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, *Real-Time Rendering*, 4th ed. CRC Press, 2018.

[3] B. T. Phong, "Illumination for computer generated pictures," *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.

[4] J. F. Blinn, "Models of light reflection for computer synthesized pictures," *Computer Graphics (Proc. SIGGRAPH '77)*, vol. 11, no. 2, pp. 192–198, 1977.

[5] H. Gouraud, "Continuous shading of curved surfaces," *IEEE Transactions on Computers*, vol. C-20, no. 6, pp. 623–629, 1971.

[6] T. Whitted, "An improved illumination model for shaded display," *Communications of the ACM*, vol. 23, no. 6, pp. 343–349, 1980.

[7] L. Williams, "Casting curved shadows on curved surfaces," *Computer Graphics (Proc. SIGGRAPH '78)*, vol. 12, no. 3, pp. 270–274, 1978.

[8] W. T. Reeves, D. H. Salesin, and R. L. Cook, "Rendering antialiased shadows with depth maps," *Computer Graphics (Proc. SIGGRAPH '87)*, vol. 21, no. 4, pp. 283–291, 1987.

[9] C. Schlick, "An inexpensive BRDF model for physically-based rendering," *Computer Graphics Forum*, vol. 13, no. 3, pp. 233–246, 1994.

[10] E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design*, R. E. Barnhill and R. F. Riesenfeld, Eds. Academic Press, 1974, pp. 317–326.

[11] W. T. Reeves, "Particle systems—a technique for modeling a class of fuzzy objects," *ACM Transactions on Graphics*, vol. 2, no. 2, pp. 91–108, 1983.

[12] I. Quilez, "Ray–surface intersectors." https://iquilezles.org/articles/intersectors/

[13] D. Hoskins, "Hash without sine," Shadertoy. https://www.shadertoy.com/view/4djSRW

[14] "Tileable water caustic," Shadertoy. https://www.shadertoy.com/view/MdlXz8

[15] K. Narkowicz, "ACES filmic tone mapping curve," 2016. https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/

[16] J. de Vries, *Learn OpenGL*. https://learnopengl.com

[17] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification (Version 3.3, Core Profile)*. Khronos Group, 2010.

[18] J. Kessenich, *The OpenGL Shading Language, Version 3.30*. Khronos Group, 2010.

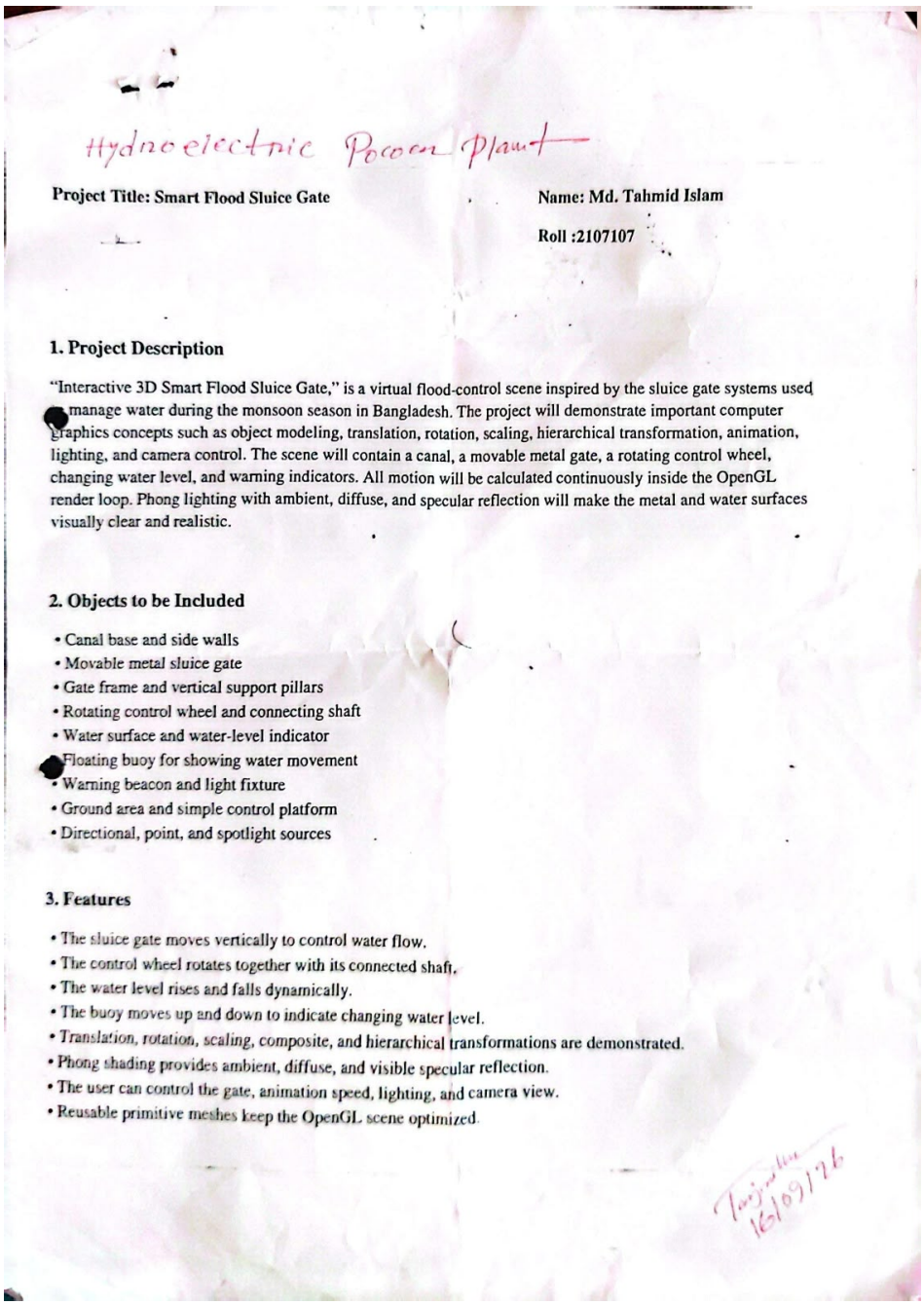
[19] F. M. White, *Fluid Mechanics*, 7th ed. McGraw-Hill, 2011.

[20] S. J. Chapman, *Electric Machinery Fundamentals*, 5th ed. McGraw-Hill, 2012.

Appendix A

The Approved Proposal

Figure A.1 shows the original proposal. The teacher's note "Hydroelectric Power Plant" at the top asked for the sluice-gate idea to be grown into a hydroelectric power plant simulation, which is the project described in this report.



Hydroelectric Power Plant

Project Title: Smart Flood Sluice Gate

Name: Md. Tahmid Islam

Roll :2107107

1. Project Description

"Interactive 3D Smart Flood Sluice Gate," is a virtual flood-control scene inspired by the sluice gate systems used to manage water during the monsoon season in Bangladesh. The project will demonstrate important computer graphics concepts such as object modeling, translation, rotation, scaling, hierarchical transformation, animation, lighting, and camera control. The scene will contain a canal, a movable metal gate, a rotating control wheel, changing water level, and warning indicators. All motion will be calculated continuously inside the OpenGL render loop. Phong lighting with ambient, diffuse, and specular reflection will make the metal and water surfaces visually clear and realistic.

2. Objects to be Included

- Canal base and side walls
- Movable metal sluice gate
- Gate frame and vertical support pillars
- Rotating control wheel and connecting shaft
- Water surface and water-level indicator
- Floating buoy for showing water movement
- Warning beacon and light fixture
- Ground area and simple control platform
- Directional, point, and spotlight sources

3. Features

- The sluice gate moves vertically to control water flow.
- The control wheel rotates together with its connected shaft.
- The water level rises and falls dynamically.
- The buoy moves up and down to indicate changing water level.
- Translation, rotation, scaling, composite, and hierarchical transformations are demonstrated.
- Phong shading provides ambient, diffuse, and visible specular reflection.
- The user can control the gate, animation speed, lighting, and camera view.
- Reusable primitive meshes keep the OpenGL scene optimized.

Approved 16/09/26

Figure A.1: The original proposal, *Smart Flood Sluice Gate*, with the teacher's note.

Appendix B

Building and Running the Program

Source code

The complete source code of the project is available on GitHub:
https://github.com/tahmidalbi/Hydro-electric-Powerplant-Simulation-with-OpenGL

Requirements

Windows with MSYS2 MinGW-w64 installed at `C:\msys64\mingw64` (including the GLFW package), and a graphics card with OpenGL 3.3.

Building and running

- Double-click `run.bat`. It builds the program the first time and then starts it.
- Or open a terminal inside the project folder. `build.bat` compiles the program, and `run.bat` starts it. The program itself is `build\hydro.exe`.

Command-line options

Option	Effect
`--size 1920 1080`	Window size
`--nopost`	Start without bloom, vignette and grain
`--noshadows`	Start with shadows off
`--shading phong|gouraud`	Start in the given shading mode
`--bench`	Print the frame time of every scene
`--shot 7 19 pic.bmp`	Start scene 7, run 19 s on AUTO, save a picture
`--autotest 60 pic.bmp`	Run the real frame loop on AUTO for 60 s, then save a picture

Project folder

```
HYDRO/
  build.bat, run.bat, CMakeLists.txt
  shaders/        21 GLSL files: lighting, water, sky, particles, ray tracer,
                  flow lines, post-processing, HUD
  src/
    main.cpp      command line
    App.cpp/.h    window, input, transitions, speed control, HUD
    core/         MathLib, Mesh + MeshBuilder, Shader, Camera, Screenshot
    render/       Renderer, ParticleSystem, FrameCapture, PostFX
    objects/      one file per object + World + PlantData
    scenes/       Scene1_Preview ... Scene11_Grid
    ui/           pixel Font, Ui
  docs/           technical notes
  captures/       pictures rendered by the program
  report/         this report
```